Text Cleaning & Pre-processing

Modulo 2 · Lezione 5



Agenda

Un solo file per tutta la serata: `working/clean.py` .

🎤 = il docente scrive, voi replicate · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Quando il rumore fa sbagliare	slide	30′
🎤	**Live coding: i primi tre step**	`working/clean.py`	30′
⌨️	**Gli step che cambiano la risposta**	`working/clean.py`	35′
📊	Al rientro: la risposta è cambiata	slide	15′
📊	Il costo, e il ponte al lab	slide	10′



Prima di cominciare: l'ambiente

Stessa struttura di **Modulo 2 · Lezione 4**: si lavora **dentro** `scripts/`, e tutti i comandi della serata si lanciano da lì. Fatelo **adesso** — l'installazione è lunga e intanto noi parliamo.

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Nel `.env` va la stessa chiave di **Modulo 2 · Lezione 1–4**.

Non ce l'avete o non funziona? **Alzate la mano**: il docente ne ha una di riserva. Se manca, lo script ve lo dice — non è colpa del vostro codice.

Un solo comando, tutta la sera:

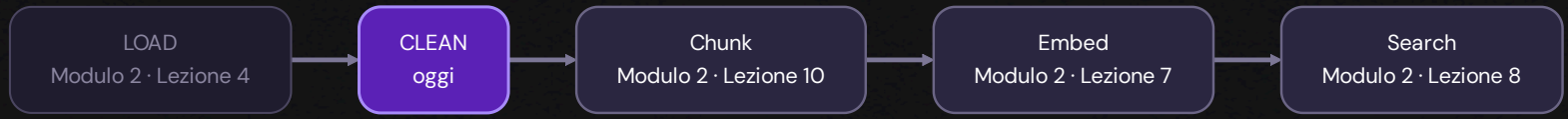
`python working/clean.py`

Sempre da `scripts/`. Gira anche a metà: stampa quello che c'è e dice cosa manca.



Dove siamo

La lezione scorsa i documenti sono **entrati**: quattro formati, un `Document`, una porta sola. Quello che è entrato però non è pulito — il contenitore è arrivato dentro insieme al contenuto.



> Stasera vediamo che quel contenitore **può far sbagliare la risposta**.

1. Quando il rumore fa sbagliare



Il documento di stasera

Il manuale della **LumaDesk**, quattro pagine, caricato con `load()` — quello che avete scritto voi la lezione scorsa. Su ogni pagina, in piccolo, un piede di pagina.

su ogni pagina, 4 volte

```
Lumen S.r.l. · LumaDesk Mini — portata piano 80 kg · lumen.it/mini
```

Pubblicità del modello gemello. Vera, ma su un **altro prodotto**.

nel capitolo 3, una volta

```
Il ripiano sopporta un carico massimo di 120 kg distribuiti in modo uniforme.
```

Il dato vero della scrivania di questo manuale.

> La domanda che facciamo al modello: **«Qual è la portata del piano?»**



La stessa domanda, due risposte

Stesso documento, stessa domanda, stesso modello. Cambia solo se il testo è stato ripulito.

	Caratteri	«80 kg» nel testo	Risposta del modello
testo estratto così com'è	2.082	4 volte	**80 kg** — sbagliata
testo ripulito	1.751	mai	**120 kg** — giusta

Il modello non si è *confuso*: ha risposto con sicurezza un numero che nel documento c'è davvero. È solo la portata di un **altro modello di scrivania**, quello pubblicizzato nel piede di pagina.



Perché il piede vince sul contenuto

Guardate dove stanno le parole della domanda — **portata** e **piano**.

IL PIEDE — 4 volte

`LumaDesk Mini — portata piano 80 kg`

Contiene **tutte e due** le parole della domanda, e le ripete quattro volte.

IL CONTENUTO — 1 volta

`Il ripiano sopporta un carico massimo di 120 kg`

Dice la stessa cosa con **altre parole**. Nessuna coincide con la domanda.

> Il modello sceglie il pezzo di testo che **somiglia di più alla domanda**. Qui è il rumore. È lo stesso modo in cui, più avanti, sbaglierà un retrieval: la risposta sbagliata usa le parole della domanda meglio di quella giusta.



Perché il loader non basta

La lezione scorsa il contorno l'avete tolto con una riga. Perché qui non funziona?

PAGINA WEB

```
<nav> … </nav>
<header> … </header>
<main> il contenuto </main>
<footer> … </footer>
```

Il contorno è **marcato**: ha un nome. Sei tag e via.

PDF ESTRATTO

```
3. Carico di lavoro
Il ripiano sopporta un carico…
Lumen S.r.l. · LumaDesk Mini…
pagina 3 di 4
```

Il piede è **testo come tutto il resto**. Non c'è niente da togliere per struttura.

Serve un'**euristica**: una riga corta che compare quattro volte è impaginazione, non contenuto.



Le famiglie di rumore

Quattro tipi, tutti presi dai tre documenti che avete in `dataset/`.

Mobilia di pagina

Intestazioni, piedi, numeri di pagina. `pagina 3 di 4` · il piede della Mini.

Impaginazione

Righe spezzate a lunghezza fissa e parole tagliate: `lit-erature`, `cor-pus`.

Boilerplate

Licenze, note legali, intestazioni di progetto. Frankenstein ne ha in testa e in coda.

Residui di formato

Spazi multipli, caratteri unicode strani, righe vuote a raffica.

> Solo il primo, stasera, ha fatto sbagliare la risposta. Gli altri costano token e leggibilità — e ogni tanto, su un altro documento, tocca a loro.



Cosa non rimuovere

La pulizia toglie il **contenitore**, non il contenuto. Chi taglia troppo perde la risposta, e non se ne accorge.

VIA

Piedi e intestazioni ripetuti · numeri di pagina · spazi doppi · righe spezzate a metà frase

RESTA

Numeri e unità di misura · codici prodotto e sigle · titoli di capitolo · tabelle

Un piede che compare **due** volte su un documento di due pagine non è distinguibile da una frase ripetuta apposta. La soglia è una scelta, e ogni tanto sbaglia.



2. Gli step



I cinque step di stasera

Prima di scriverli, la mappa. Ognuno fa una cosa sola e restituisce una stringa.

🎤 `normalize_whitespace` — spazi multipli, righe vuote di troppo, caratteri unicode strani

🎤 `dehyphenate` — riunisce le parole spezzate a fine riga

🎤 `reflow_paragraphs` — ricuce le righe interrotte dall'impaginazione

⌨️ `remove_repeated_lines` — toglie le righe corte che si ripetono ← quello che salva la risposta

⌨️ `remove_page_numbers` — toglie le righe che sono solo un numero di pagina



🎤 Live coding — i primi tre step

🎤 **LIVE CODING — si guarda e si replica**

`working/clean.py · 30 minuti`

si torna alle slide dopo `reflow_paragraphs`

TODO 1 · `normalize_whitespace`

Il più meccanico: si parte da qui per prendere la forma.

fatto quando: il conteggio caratteri scende

TODO 2 · `dehyphenate`

Una regex sola, e le parole tornano intere.

fatto quando: le **3** parole col trattino tornano intere

TODO 3 · `reflow_paragraphs`

Il primo che ragiona sul testo: quando è finita una frase?

fatto quando: le righe crollano da 38 a ~9

`python working/clean.py` (da `scripts/`) dopo **ogni** step: stampa quello che c'è e dice quali mancano. Non serve finire tutto per vedere qualcosa.



Ogni step è una funzione normale

Prende una stringa, restituisce una stringa. Le ricette in fondo al file le mettono in fila: si legge dall'alto in basso, senza saltare.

```
def normalize_whitespace(text):
    """Spazi multipli e righe vuote di troppo → uno spazio, una riga."""
    ...
    return text


def pulisci_txt(text):
    """Un libro in testo semplice: spaziatura irregolare, righe a lunghezza fissa."""
    text = normalize_whitespace(text)
    text = reflow_paragraphs(text)
    return text
```

> `pulisci_txt` è già scritta nel file: guardatela, è il modello per la vostra. **L'ordine conta**: se togliete i numeri di pagina *dopo* aver ricucito le righe, i numeri sono già finiti in mezzo alle frasi.



`reflow_paragraphs` sul manuale

L'estrazione da PDF spezza le frasi dove finiva la riga sulla pagina. Ricucirle è l'effetto più visibile della serata.

PRIMA — estratto · tutto il documento: 38 righe

```
Il ripiano sopporta un carico massimo di 120 kg
distribuiti in modo
uniforme sulla superficie. Il peso va disposto in modo
simmetrico
rispetto alle gambe: un carico concentrato su un solo
lato riduce
```

DOPO — estratto · tutto il documento: 9 righe

```
Il ripiano sopporta un carico massimo di 120 kg
distribuiti in modo uniforme sulla superficie. Il peso
va disposto in modo simmetrico rispetto alle gambe: un
carico concentrato su un solo lato riduce
```

Una riga che non finisce con un punto non è finita: continua in quella dopo. Su Frankenstein lo stesso step porta **7.741 righe a 1.098**.



`dehyphenate` sul paper arXiv

Nei testi giustificati le parole si spezzano a fine riga. Nel manuale succede **3 volte** (`colle-gamento` , `imme-diata` , `dia-gnostica`): le vedete sparire nell'output. **Demo del docente** su un altro documento del `dataset/` , `arxiv_1905.00075.pdf` , dove succede **15 volte** su parole che contano. Voi non toccate nulla adesso: se volete rifarlo, è fra gli extra di fine esercizio.

COME ESCE DAL PDF

```
lit-
erature

cor-
pus

au-
thorship
```

DOPO

```
literature

corpus

authorship
```

> Qui il modello se la cava lo stesso: legge attraverso il trattino. Ma **una ricerca no** — cercare «corpus» non trova «cor-pus». Ci torneremo quando cercheremo davvero.



La pulizia è euristica, non esatta

«Una riga corta che compare almeno tre volte è impaginazione» non è una verità: è una **scelta**, tarata su questi documenti.

SE ALZO LA SOGLIA A 5

Il piede del manuale compare 4 volte: **non viene più tolto**, e la risposta torna a essere 80 kg.

SE LA ABBASSO A 2

Sparisce anche una frase che l'autore ha ripetuto **apposta** — un avvertimento di sicurezza, per esempio.

Su un corpus nuovo la soglia si ritara, guardando cosa esce.



⌨️ Tocca a voi — gli step che cambiano la risposta

⌨️ **SCRIVETE VOI — il docente affianca**

`working/clean.py` · 35 minuti

si riapre
lo stesso file

TODO 4 · `remove_repeated_lines`

Conta le righe, togli quelle corte che si ripetono. **Quante** ripetizioni e **quanto** corta le decidete voi: il main stampa «LumaDesk Mini: Nx», usatelo per tarare.

TODO 5 · `remove_page_numbers`

Le righe che sono *solo* un numero.

TODO 6 · `pulisci_pdf`

Mettete i cinque step in fila, nell'ordine che vi convince.

Avete finito quando `python working/clean.py` (da `scripts/`) arriva in fondo e la riga «sul testo PULITO» dice **120 kg**.

Finito prima? Cambiate `DOCUMENTO` e provate su `frankenstein.txt` o `arxiv_1905.00075.pdf` — quale ricetta serve a ciascuno? Oppure alzate la soglia delle ripetizioni da 3 a 5 e guardate tornare la risposta sbagliata.
Rilanciate quanto volete: un giro costa **$0,0006**.



📊 Al rientro — la risposta è cambiata

Le due righe che stampa il vostro `clean.py`, con il vostro codice, sulla vostra macchina.

```
domanda: Qual è la portata del piano?
sul testo SPORCO : La portata del piano è di 80 kg.
sul testo PULITO : La portata del piano è di 120 kg distribuiti in modo uniforme.
```

> Non avete cambiato il modello, né il prompt, né la domanda. Avete tolto **325 caratteri** di mobilia di pagina.



Ricette diverse per documenti diversi

Un PDF impaginato e un libro in testo semplice hanno rumore diverso: si mettono in fila step diversi.

`pulisci_pdf`

```
remove_repeated_lines
remove_page_numbers
dehyphenate
normalize_whitespace
reflow_paragraphs
```

Il manuale: **2.082 → 1.751** caratteri

`pulisci_txt`

```
normalize_whitespace
reflow_paragraphs
```

`frankenstein.txt`, l'altro documento del `dataset/`: **7.741 → 1.098** righe

> Non c'è una pulizia universale: si scrive una ricetta per famiglia di documenti, guardando cosa esce.



Il costo, come effetto collaterale

Sul manuale la pulizia toglie il **15,9%** dei caratteri. Su un documento sono spiccioli. Il conto cambia quando i documenti sono diecimila e le domande sono continue.

1	100	10.000
documento	documenti	documenti × N domande
325 caratteri in meno	un capitolo intero di rumore	è la bolletta

> Ma il motivo per cui stasera abbiamo pulito non è il costo: è che **la risposta era sbagliata**. Il risparmio è venuto dietro.



Il testo è pulito, ma è un blocco unico

Il manuale ripulito sono ancora 1.751 caratteri in un pezzo solo. Per rispondere a una domanda su una riga, il modello se lo rilegge tutto — ogni volta.

Il prossimo lab passa ai documenti **veri**: PDF scansionati, tabelle che si spappolano, encoding sbagliati, file che il loader non riesce nemmeno ad aprire. Un lab di quattro ore su dati sporchi come li trovate al lavoro.

> Prossimo passo: **Modulo 2 · Lezione 6 — Lab Data Engineering**.



Grazie

Prossima lezione: Lab Data Engineering (Modulo 2 · Lezione 6)

